

# **2024 Sept OPPE 1 - SET 2**

## **Section 3 - Problem 1**

# Problem Statement

## Problem statement:

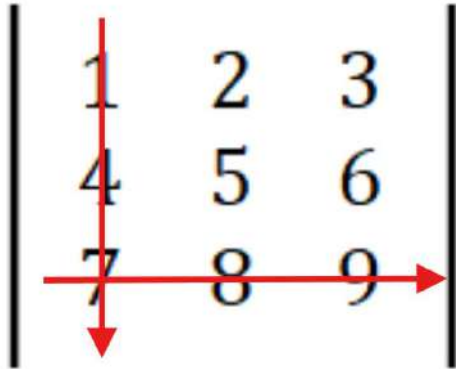
- You are given a square matrix and a path type. Your task is to traverse the matrix according to the specified path and return the elements in the order of traversal.
- **Path Types:**
  - **"L"**: Start at the top-left corner and traverse in an L-shape (first column down, then last row right).
  - **"Z"**: Start at the top-left corner and traverse in a Z-shape (top row left→right, diagonal down-right, bottom row left→right).
  - **"O"**: Start at the top-left corner and traverse clockwise around the outer edge of the matrix.
- **Constraints:**
  - Square matrix of size  $n \times n$ .

# Example and Visualisation

We have a 3 by 3 matrix:

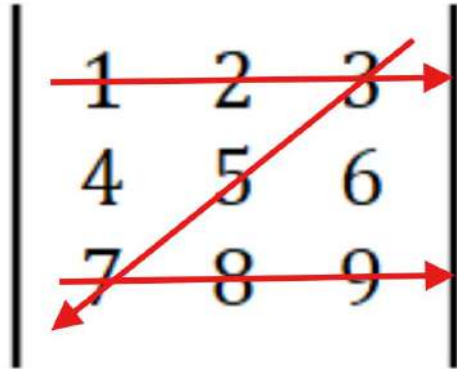
$$A = \begin{vmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{vmatrix}$$

L shape Traversal



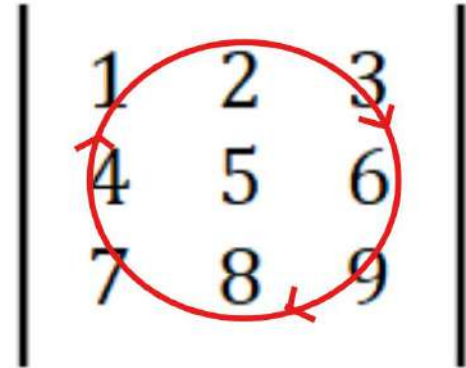
Order of traversal:  
1 4 7 8 9

Z shape Traversal



Order of traversal:  
1 2 3 5 7 8 9

O shape Traversal



Order of traversal:  
1 2 3 6 9 8 7 4

# Logic Visualization of our Problem

## Identify the Path and Matrix Size

Usage of if - elif statements

Determine which path is requested: "L", "Z", or "O".

Get the size of the square matrix  $n$  to control the loop ranges.

This ensures that the traversal logic adapts correctly to any  $n \times n$  matrix.

## Traverse According to Path Rules

Compute logic for each traversal under their if statements

- **L-shape:** Traverse the first column top  $\rightarrow$  bottom, then the last row left  $\rightarrow$  right.
- **Z-shape:** Traverse the top row left  $\rightarrow$  right, diagonal top-right  $\rightarrow$  bottom-left, and bottom row left  $\rightarrow$  right.
- **O-shape:** Traverse the outer edge clockwise: top row, last column, bottom row reversed, first column reversed.

## Collect Elements in Order

`list.append(element)`

- During traversal, append each visited element to a **result list**.
- Maintain the order as per the path so that the final list matches the expected traversal sequence.

## Return the Result

Result: List with elements in their traversal sequence

- After the traversal is complete, return the **result list**.
- The list contains all elements in the exact order of the specified path.

# Mathematical Reasoning

## Our Sample Matrix: 3×3 Grid Structure

We'll use the following 3×3 matrix (M) to demonstrate all three traversal patterns. Each element has a unique position and value.

|  $M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$

1

Position notation:  
 $M[\text{row}][\text{column}]$   
where indices start  
at 0

2

Row 0: Elements 1, 2,  
3 (top row)

3

Row 1: Elements 4, 5,  
6 (middle row)

4

Row 2: Elements 7,  
8, 9 (bottom row)

# Traversal Logic

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |

## L-Shape Code Implementation

### Summary

Two separate loops: first loop collects the first column; second loop collects the last row (skipping the corner we already got)

### Code

```
if path == "L": --> for i in range(n): --> result.append(matrix[i][0])
```

```
Last row right (excluding first element) = for j in range(1, n) --> result.append(matrix[n-1][j])
```

#### 1 Loop 1: Column

range(n) generates 0, 1, 2...  
Variable i is the row index.  
matrix[i][0] always accesses  
column 0 (first column)

#### 2 Loop 2: Row

range(1, n) generates 1, 2...  
starting from 1 to skip first  
element. matrix[-1][j]  
accesses the last row (-1 is  
last index)

#### 3 Why Exclude?

Starting from j=1 prevents  
adding matrix [-1][0] twice,  
since it was already added  
by the first loop

# Traversal Logic

## Z-Shape Code Implementation

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |

### Summary

Three separate loops: top row, then diagonal, then bottom row

### Code

```
elif path == "Z":
```

```
# top row --> for j in range(n): result.append(matrix[0][j])
```

```
# diagonal (excluding first and last) --> for i in range(1, n-1): result.append(matrix[i][n-i-1])
```

```
# bottom row --> for j in range(n): result.append(matrix[n-1][j])
```

1

### Loop 1: Top Row

matrix[0][j] keeps row at 0, j varies across columns.  
range(n) gives us all columns: 0, 1, 2

2

### Loop 2: Diagonal

The formula  $n-i-1$  calculates the column index. range(1, n-1) gives middle rows only (1).  
When  $i=1$ : column =  $3-1-1 = 1$  (middle element)

3

### Loop 3: Bottom

matrix[n-1][j] keeps row at last index. j varies to get all columns in bottom row

# Traversal Logic

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |

## O-Shape Code Implementation (Part 1 of 2)

### Summary

Four separate loops traverse the outer border clockwise: top → right → bottom → left

### Code (first two loops)

```
elif path == "O":
```

```
# top row --> for j in range(n): result.append(matrix[0][j])
```

```
# right column (excluding top) --> for i in range(1, n): result.append(matrix[i][n-1])
```

1

2

### Loop 1: Top Row →

`matrix[0][j]` with `j` in `range(n)` traverses the entire top row from left to right, collecting all elements [1, 2, 3]

### Loop 2: Right Column ↓

`matrix[i][n-1]` keeps column at last index (`n-1`). `range(1, n)` starts at 1 to skip the top-right corner already collected, going down [6, 9]



# Traversal Logic

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |

## O-Shape Code Implementation (Part 2 of 2)

### Summary

Completing the clockwise border: bottom row moves right-to-left; left column moves bottom-to-top

```
# bottom row (excluding last) --> for j in range(n-2, -1, -1): result.append(matrix[n-1][j])
```

```
# left column (excluding bottom and top) --> for i in range(n-2, 0, -1): result.append(matrix[i][0])
```

1

2

### Loop 3: Bottom Row ←

`range(n-2, -1, -1)` creates reverse sequence: 1, 0 (right to left). Starts at `n-2` to skip bottom-right corner. `matrix[n-1][j]` accesses last row [8, 7]

### Loop 4: Left Column ↑

`range(n-2, 0, -1)` generates 1 (just middle row going up). Excludes 0 and `n-1` to avoid re-adding corners. `matrix[i][0]` accesses first column [4]

# Key Concepts Used in this Challenging Problem



## **range()** **Function**

`range(n)` generates 0 to  $n-1$ . `range(1, n)` generates 1 to  $n-1$ . `range(n-2, -1, -1)` generates reverse from  $n-2$  down to 0. Third parameter is the step direction



## **Matrix** **Indexing**

`matrix[i][j]` where  $i$ =row,  $j$ =column. `matrix[0][j]` means first row, any column. `matrix[i][0]` means any row, first column.  $n-1$  always refers to last index



## **Exclusion Logic**

Starting ranges at 1 instead of 0 skips first element. Using  $n-1$  or  $n-2$  as start/end prevents duplicate corner elements. This avoids adding the same element twice



## **result.append()** **)**

Adds each element to the result list in order. The sequence of loops determines the final traversal order. Each append builds the output step by step

# Solution 1: The Explanatory Approach (I)

```
def matrix_walk(matrix, path):  
    def l_shape():  
        res = [matrix[i][0] for i in range(len(matrix))]  
        res += [matrix[-1][j] for j in range(1, len(matrix))]  
        return res  
  
    def z_shape():  
        n = len(matrix)  
        res = [matrix[0][j] for j in range(n)]  
        res += [matrix[i][n-i-1] for i in range(1, n-1)]  
        res += [matrix[-1][j] for j in range(n)]  
        return res
```

**The most beginner friendly way to solve this problem**

# Solution 1: The Explanatory Approach (II)

```
def o_shape():
    n = len(matrix)
    res = [matrix[0][j] for j in range(n)]
    res += [matrix[i][n-1] for i in range(1, n)]
    res += [matrix[-1][j] for j in range(n-2, -1, -1)]
    res += [matrix[i][0] for i in range(n-2, 0, -1)]
    return res

if path == "L":
    return l_shape()
elif path == "Z":
    return z_shape()
elif path == "O":
    return o_shape()
```

**The most beginner friendly way to solve this problem**

## Solution 2: Quicker and Shorter version

```
def matrix_walk(matrix, path):  
    n = len(matrix)  
    if path == "L":  
        return [matrix[i][0] for i in range(n)] + [matrix[n-1][j] for j in  
range(1, n)]  
    elif path == "Z":  
        return [matrix[0][j] for j in range(n)] + [matrix[i][n-i-1] for i in  
range(1, n-1)] + [matrix[n-1][j] for j in range(n)]  
    elif path == "O":  
        return [matrix[0][j] for j in range(n)] + [matrix[i][n-1] for i in  
range(1,n)] + [matrix[n-1][j] for j in range(n-2,-1,-1)] + [matrix[i][0] for  
i in range(n-2,0,-1)]
```

**Combining multiple for loop statements using List Comprehensions and Adding Lists**

# Solution comparison

| Approach                                            | Pros                                                                    | Cons                                                               | Best For                                           |
|-----------------------------------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------------|----------------------------------------------------|
| Explanatory Approach                                | Highly readable, easy to debug, best for learning, clear variable names | More lines of code, multiple variables                             | Educational purposes, beginners, complex debugging |
| Multiple List Comprehensions in a single expression | Most concise, Pythonic, demonstrates mastery                            | Harder to debug, less readable for beginners, repeated calculation | Experienced developers, simple contexts            |

# Key Takeaways

- 🔑 **Matrix Indexing** (`matrix[i][j]`) is essential for accessing elements in specific rows and columns.
- 🔑 **Iteration/Loops** allow traversing rows, columns, and diagonals systematically.
- 🔑 **Conditional Logic** (if-elif-else) is used to select the path dynamically.
- 🔑 **List Construction** collects elements as per the traversal pattern.
- 🔑 **Helper Functions** like List Comprehensions can improve readability and separate logic for different paths.
- 🔑 **Understanding row and column ranges** is key for correctly implementing L, Z, and O shapes.